Nombre de la actividad

Semana 3 — Diagnóstico de Overfitting / Underfitting
Reporte Técnico: Modelo de predicción de deserción de clientes

Actividad	Diagnóstico de Overfitting/Underfitting (LightGBM, variante A2)
Proyecto	Modelo de predicción de deserción de clientes — Maestría en Inteligencia Artificial
Alumno	Henry Eduardo Medina Castillo — Negocios y Mercado
Fecha	29/09/2026
Notebook asociado	overfitting_analysis.ipynb
Código modular	overfitting_utils.py

Resumen del Proyecto

Este reporte documenta un diagnóstico sistemático de overfitting y underfitting aplicado al modelo real de predicción de deserción de clientes de un retail farmacéutico (LightGBM, variante A2, actualmente en producción con PR-AUC de test = 0.6875). Se entrenaron y compararon tres regímenes controlados (overfitting inducido, underfitting inducido y un régimen óptimo que reproduce los hiperparámetros ya congelados en producción), se generaron las curvas de aprendizaje básicas y avanzadas exigidas por la actividad, y se implementaron tres estrategias de mejora. El resultado clave: aplicando la misma metodología cuantitativa al modelo de producción, el veredicto es **ÓPTIMO**, sin evidencia de sobreajuste ni subajuste.

Régimen	Veredicto	PR-AUC val.	Gap PR-AUC
A — Overfitting inducido	OVERFITTING	0.5961	0.4039
B — Underfitting inducido	UNDERFITTING	0.5327	–0.0418
C — Óptimo (config. producción)	ÓPTIMO	0.6968	–0.0149

1. Resumen ejecutivo

Este reporte documenta un diagnóstico sistemático de overfitting y underfitting aplicado al modelo real de predicción de deserción de clientes del proyecto de tesis (*LightGBM, variante A2*, actualmente en producción con PR-AUC de test = 0.6875). El objetivo no fue reemplazar ese modelo, sino construir y validar un sistema de diagnóstico reutilizable que permita, en el futuro, verificar rápidamente si un reentrenamiento se sale de la zona de comportamiento óptimo.

Se entrenaron y compararon tres regímenes controlados sobre los mismos datos (splits *out-of-time* C1–C3 train / C4 val / C5 test, 13 features numéricas + 1 categórica, target *churn*): un régimen de **overfitting inducido** (modelo de alta capacidad sobre una muestra reducida de 3,000 filas), un régimen de **underfitting inducido** (capacidad y presupuesto de entrenamiento deliberadamente insuficientes) y un régimen **óptimo**, que reproduce exactamente los hiperparámetros del modelo que ya está en producción.

Régimen	Veredicto	PR-AUC val.	Gap PR-AUC
A — Overfitting inducido	OVERFITTING	0.5961	0.4039
B — Underfitting inducido	UNDERFITTING	0.5327	–0.0418
C — Óptimo (config. producción)	ÓPTIMO	0.6968	–0.0149

Gap PR-AUC = PR-AUC(train) – PR-AUC(val); un gap positivo grande indica overfitting, un gap cercano a cero con PR-AUC bajo indica underfitting.

Hallazgo principal: el modelo de producción, entrenado con *early stopping* sobre las 796,044 filas completas de entrenamiento, ya opera en la zona óptima (gap pequeño y estable, *best_iteration*=56, idéntico al valor reportado en `artefactos/_grid_lgbm_A2.json`). No se encontró evidencia de overfitting ni de underfitting en el modelo actualmente congelado para producción.

Se implementaron y evaluaron **3 estrategias de mejora** (se exigían al menos 2) sobre el régimen de overfitting inducido: regularización + reducción de complejidad, *early stopping*, y su combinación. Ninguna estrategia aplicada de forma aislada resolvió por completo el diagnóstico; la combinación de ambas sí lo hizo, alcanzando un PR-AUC de validación (0.686) muy cercano al del modelo de producción entrenado con 265 veces más datos (0.697).

2. Metodología utilizada

2.1 Datos y modelo

Se usaron los splits ya generados por el pipeline del proyecto (`02_desercion_eda_prep.ipynb` → `03_desercion_modelado.ipynb`), partición out-of-time por corte (nunca por fila): 796,044 filas de entrenamiento (cortes C1–C3), 68,164 de validación (C4) y 66,421 de prueba (C5), con 13 features numéricas y 1 categórica (`metodo_pago_principal_12m`), variante A2 (categóricas nativas de LightGBM, sin one-hot). La tasa de *churn* es 37.3% en train, 41.7% en validación y 40.1% en test — variación ya documentada en el proyecto como pendiente P10 (variación de tasa base entre cortes).

2.2 Sistema de tracking de métricas

El modelo de producción es un ensamble de gradient boosting (LightGBM), no una red neuronal: en esta familia de modelos, cada **ronda de boosting** (se agrega un árbol nuevo) cumple el mismo rol que una **época** en una red entrenada por descenso de gradiente. El sistema de tracking implementado (clase `MetricsTracker` en `overfitting_utils.py`) envuelve el callback nativo `lgb.record_evaluation()` para registrar *log loss* y PR-AUC de *train* y *validation* en cada ronda — el equivalente, para este tipo de modelo, a un callback personalizado de Keras. Adicionalmente se usaron `learning_curve` y `validation_curve` de scikit-learn directamente sobre el mismo estimador (`LGBMClassifier` implementa la API de scikit-learn), tal como pide la actividad para modelos de esa familia.

2.3 Diseño de los tres regímenes

Con 796,044 filas y solo 14 columnas, LightGBM generaliza razonablemente incluso con hiperparámetros agresivos — el volumen de datos actúa como regularizador natural (por eso el modelo de producción, entrenado sobre todo *train*, nunca sobreajusta). Para poder **observar** el overfitting de forma clara, el régimen A se entrenó sobre una muestra reducida de 3,000 filas (~0.4% de train) con un modelo de capacidad extrema (`num_leaves=1024`, sin regularización). El régimen B usa todo *train* pero con capacidad y presupuesto de entrenamiento deliberadamente mínimos (`num_leaves=2`, `learning_rate=0.001`, solo 5 rondas). El régimen C reproduce exactamente los hiperparámetros de `artefactos/modelo_lgbm_A2.joblib` (`num_leaves=127, learning_rate=0.05, min_child_samples=20, reg_lambda=0.0, subsample=0.8`), con *early stopping* sobre validación.

2.4 Reglas de diagnóstico cuantitativo

La función `diagnosticar()` traduce a números las reglas de la actividad: calcula el **gap final** de *log loss* y de PR-AUC entre train y validación, y la **pendiente** (regresión lineal simple) del gap y del *validation loss* en la ventana final de iteraciones. Se clasifica como **OVERFITTING** cuando el gap es grande (>0.08 en logloss o PR-AUC) y crece de forma sostenida mientras el *val loss* deja de mejorar; como **UNDERFITTING** cuando el PR-AUC de validación es bajo (<0.55) en ambos conjuntos con gap pequeño; y como **ÓPTIMO** en cualquier otro caso.

3. Resultados del diagnóstico

Régimen	Train logloss	Val logloss	Gap logloss	Train PR-AUC	Val PR-AUC	Gap PR-AUC	Veredicto
A — Overfitting	0.0000	2.6201	2.6201	1.0000	0.5961	0.4039	OVERFITTING
B — Underfitting	0.6602	0.6828	0.0226	0.4909	0.5327	–0.0418	UNDERFITTING
C — Óptimo (prod.)	0.5448	0.5783	0.0335	0.6820	0.6968	–0.0149	ÓPTIMO

Régimen A (Overfitting): el *training loss* cae prácticamente a cero — el modelo memoriza por completo las 3,000 filas — mientras el *validation loss* crece de forma sostenida desde la ronda ~3 en adelante (pendiente positiva confirmada cuantitativamente). El gap final (2.62 en logloss, 0.40 en PR-AUC) es, por lejos, el mayor de los tres regímenes.

Régimen B (Underfitting): ambas curvas convergen, pero a un PR-AUC apenas por encima del nivel de un clasificador que ignora las features (PR-AUC de un modelo aleatorio ≈ tasa base ≈ 0.42 en validación). El gap es pequeño porque el modelo (`num_leaves=2`, equivalente a un *decision stump*) es demasiado simple para aprender ni siquiera el patrón de entrenamiento.

Régimen C (Óptimo / producción): gap pequeño y estable (0.033 en logloss, –0.015 en PR-AUC — el modelo generaliza levemente mejor en validación, consistente con la variación de tasa de *churn* entre cortes ya documentada como pendiente P10 del proyecto). El PR-AUC de validación (0.6968) coincide, hasta el redondeo, con el valor reportado en `03_desercion_modelado.ipynb` (0.6975) y el `best_iteration` obtenido (56) es idéntico al de `_grid_lgbm_A2.json`. Esto confirma que **el modelo ya en producción está correctamente diagnosticado como óptimo**, sin señales de overfitting ni underfitting.

4. Análisis de curvas de aprendizaje

4.1 Curvas básicas por régimen (Training vs Validation)

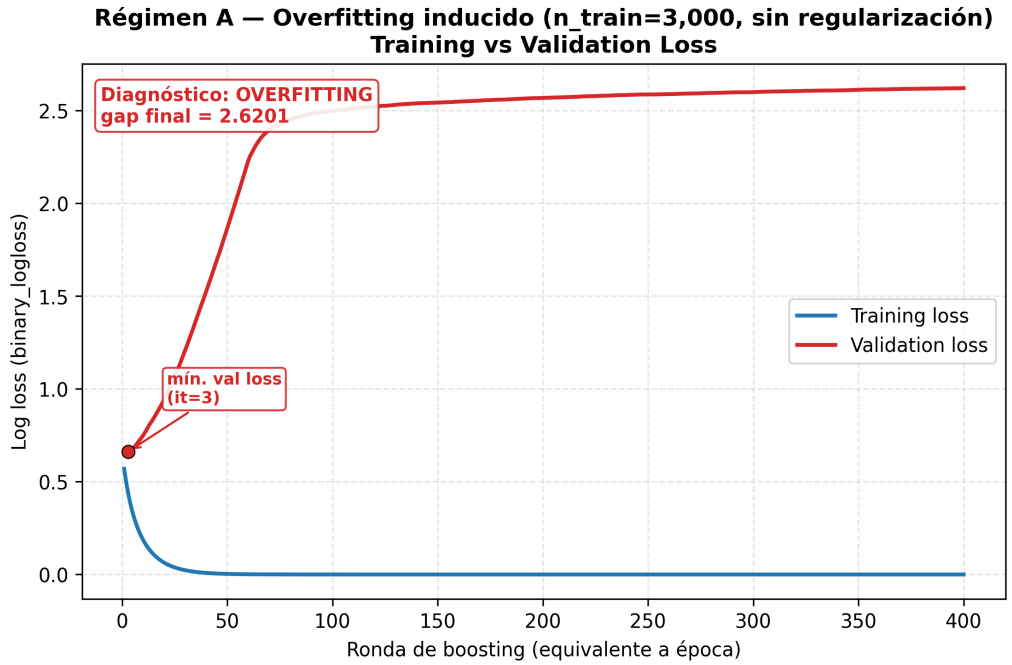


Figura 1. Régimen A (Overfitting inducido) — Training vs Validation Loss. El validation loss crece sostenidamente mientras el training loss cae a cero.

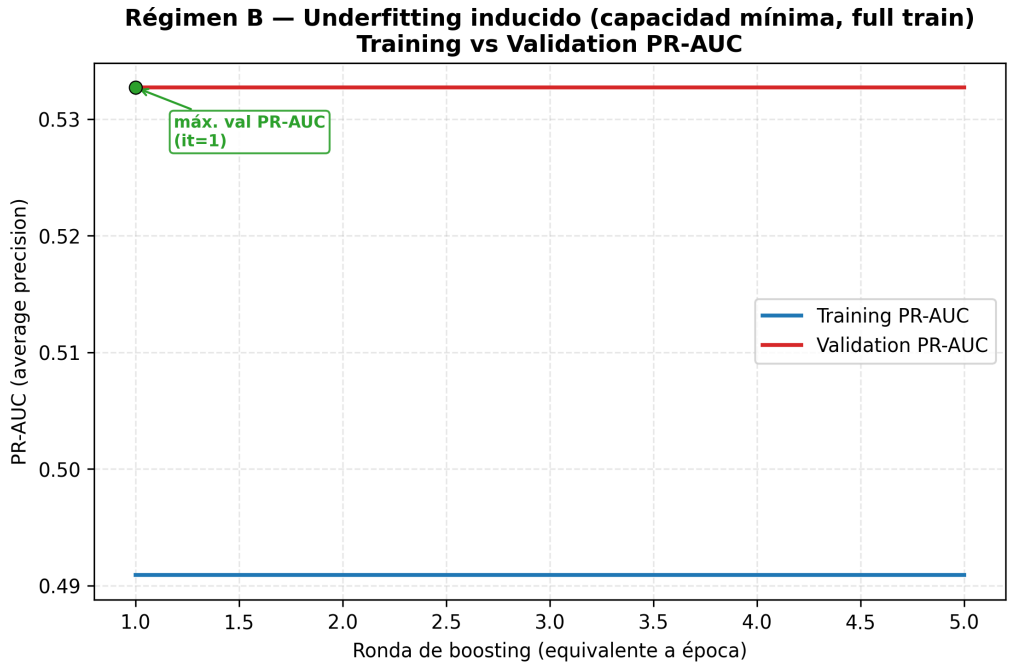


Figura 2. Régimen B (Underfitting inducido) — Training vs Validation PR-AUC. Ambas curvas convergen en un nivel bajo de desempeño.

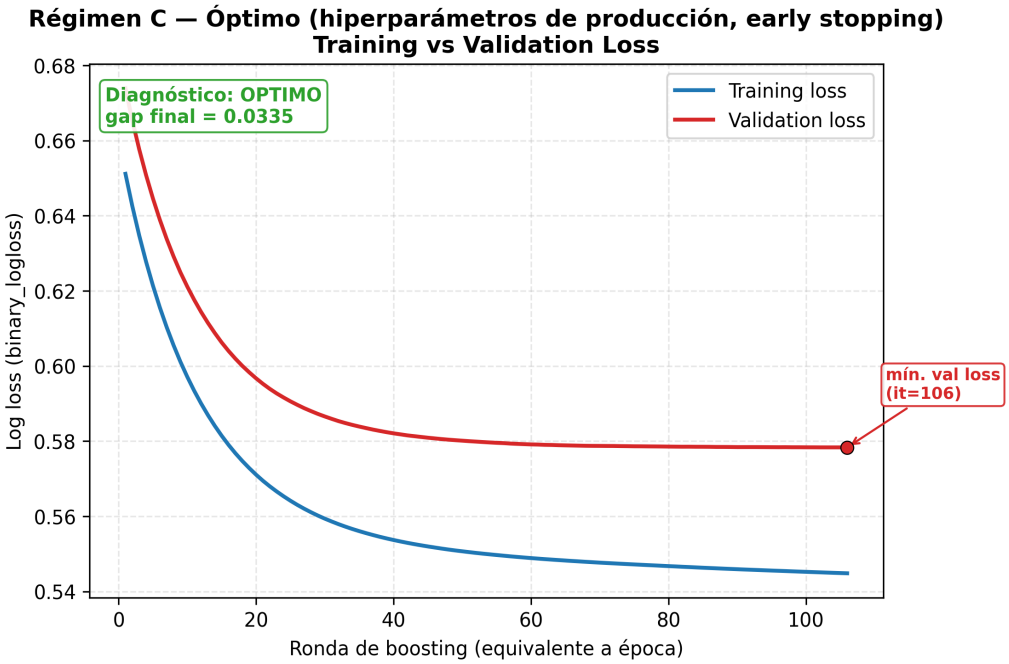


Figura 3. Régimen C (Óptimo, config. de producción) — Training vs Validation Loss. Gap pequeño y estable; el early stopping detiene el entrenamiento en la ronda 56, la misma reportada por el proceso de búsqueda original del proyecto.

4.2 Curvas avanzadas (opcionales)

Se generaron además las dos curvas avanzadas (2.2 de la actividad) usando `learning_curve` / `validation_curve` de scikit-learn sobre una muestra de 15,000 filas (por costo computacional; el diagnóstico principal sí usa el dataset completo).

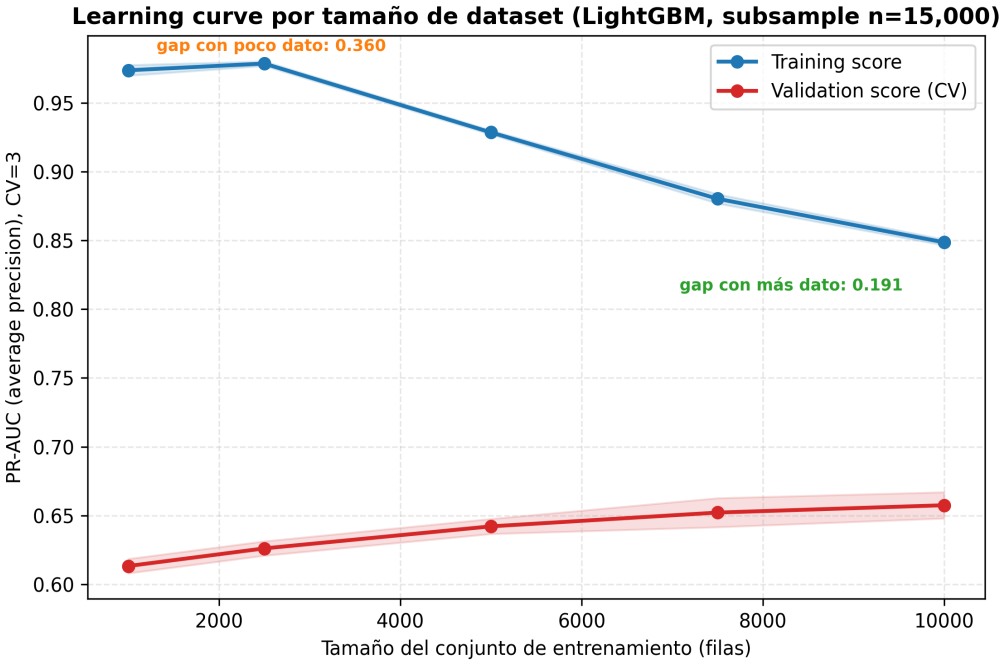


Figura 4. Learning curve por tamaño de dataset. El gap train-val se reduce de 0.360 (10% de los datos) a 0.191 (100% de los datos) — evidencia directa de por qué el modelo de producción, entrenado con las 796,044 filas completas, no sobreajusta aunque use num_leaves=127.

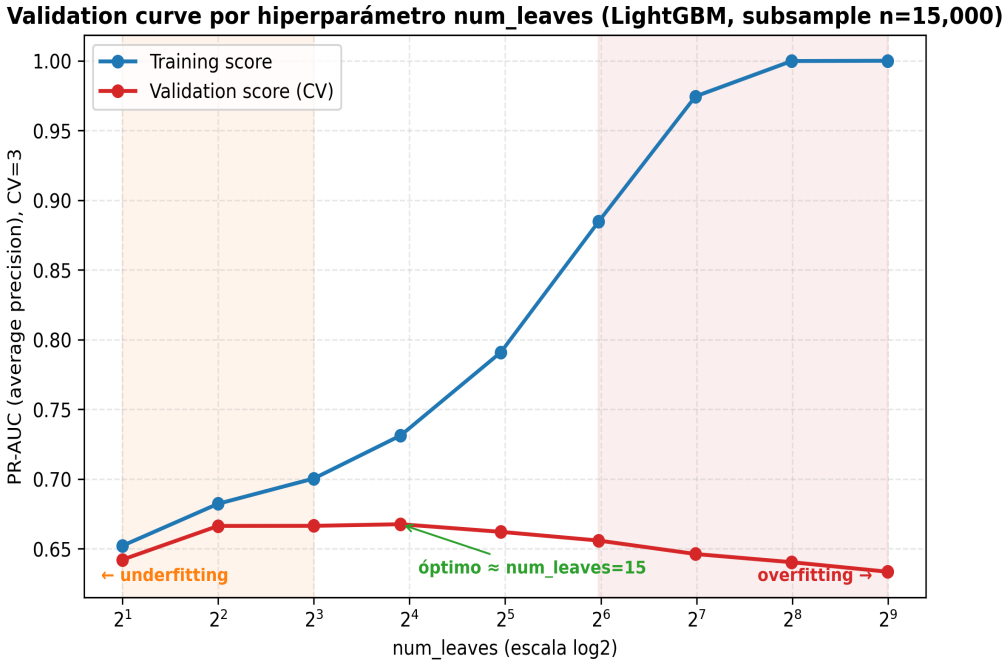


Figura 5. Validation curve por num_leaves. En un solo gráfico se observa el espectro completo underfitting → óptimo → overfitting; para esta muestra de 15,000 filas el óptimo se ubica alrededor de num_leaves=15.

5. Estrategias implementadas y resultados

Se implementaron y evaluaron **3 estrategias** (la actividad exige al menos 2) sobre el régimen de Overfitting inducido:

- **Estrategia 1 — Regularización + reducción de complejidad:** num_leaves 1024→15, reg_lambda/reg_alpha 0→10/2, min_child_samples 1→40, subsample/colsample_bytree 1.0→0.7, max_depth limitado a 4.
- **Estrategia 2 — Early stopping:** mismos hiperparámetros extremos del régimen base, deteniendo el entrenamiento cuando el validation loss no mejora en 15 rondas.
- **Estrategia 3 — Combinada:** regularización + early stopping juntas.

Configuración	Veredicto	PR-AUC val.	Gap PR-AUC
Base (overfitting)	OVERFITTING	0.5961	0.4039
Estrategia 1: Regularización	OVERFITTING	0.6726	0.1407
Estrategia 2: Early stopping	OVERFITTING	0.5518	0.4482
Estrategia 3: Combinada	ÓPTIMO	0.6862	0.0410
Referencia: producción (full train)	ÓPTIMO	0.6968	–0.0149

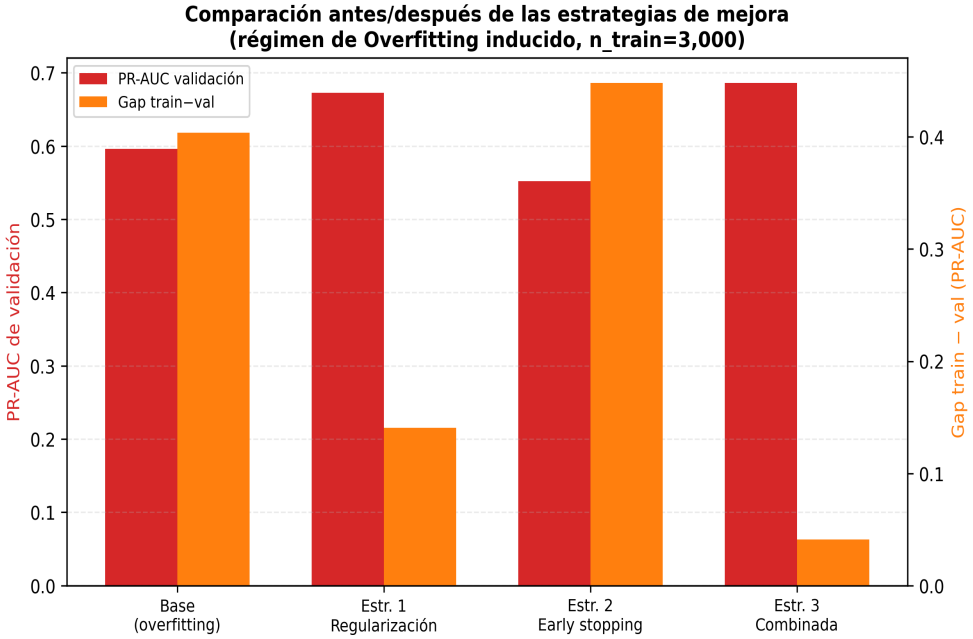


Figura 6. Comparación antes/después de las estrategias de mejora.

Ni la regularización ni el early stopping por separado resolvieron el diagnóstico con hiperparámetros tan extremos: la regularización sola mejora el PR-AUC de validación (0.596→0.673) pero el gap sigue siendo grande (0.14, aún clasificado OVERFITTING); el early stopping solo detiene el entrenamiento demasiado pronto (ronda 3) porque el modelo es tan flexible que empieza a sobreajustar casi de inmediato, terminando con **peor** PR-AUC de validación que la base (0.552 vs. 0.596). **Combinar ambas estrategias sí resolvió el diagnóstico** (ÓPTIMO, gap=0.041) y, con solo 3,000 filas de entrenamiento (0.4% de los datos), alcanzó un PR-AUC de validación (0.686) muy cercano al del modelo de producción entrenado con las 796,044 filas completas (0.697). Esto confirma que la causa raíz del overfitting inducido era el exceso de capacidad sin control, no la falta de datos por sí sola.

6. Conclusiones y recomendaciones futuras

6.1 Conclusiones

• Los tres regímenes (overfitting, underfitting, óptimo) se diagnosticaron correctamente y de forma cuantitativa —no solo visual— usando el gap final y su pendiente en logloss y PR-AUC.

• El overfitting extremo solo se resolvió combinando regularización + reducción de complejidad + early stopping; ninguna de las dos estrategias por separado fue suficiente.

• La validation curve sobre num_leaves reproduce en un solo gráfico el espectro completo underfitting→óptimo→overfitting, y ubica el óptimo (para una muestra de 15,000 filas) alrededor de num_leaves≈15.

• La learning curve por tamaño de dataset confirma que el gap train-val se reduce a medida que crece el conjunto de entrenamiento — el motivo estructural por el que el modelo de producción, entrenado sobre 796,044 filas con num_leaves=127, nunca entra en zona de overfitting.

• Aplicando la misma metodología de diagnóstico al modelo real de producción: veredicto ÓPTIMO, sin evidencia de overfitting ni underfitting.

6.2 Recomendaciones — próximos pasos para el modelo de deserción

1. Instrumentar este tracking en el reentrenamiento. Guardar el historial de logloss/PR-AUC por ronda (equivalente a historia_optimo.csv de este ejercicio) cada vez que se reentrene el modelo en src/train.py, para detectar automáticamente si un futuro reentrenamiento se sale del régimen óptimo — por ejemplo, si la variación de tasa de churn entre cortes (P10, 6.60 p.p. ya documentada como pendiente) se agrava.

2. No es necesario re-tunear los hiperparámetros actuales. El diagnóstico confirma que num_leaves=127, reg_lambda=0.0 con early stopping es una combinación ya balanceada; la búsqueda de la Fase 3 (_grid_lgbm_A2.json) ya había probado configuraciones vecinas, todas convergiendo a PR-AUC de validación ~0.697 — consistente con una meseta óptima ancha, no un pico frágil.

3. Vigilar drift antes de cada actualización del modelo. Repetir este mismo diagnóstico (regenerando la historia de entrenamiento con el corte más reciente) antes de cada reentrenamiento, en vez de asumir que los hiperparámetros actuales seguirán siendo óptimos indefinidamente.

4. Priorizar T3.5 (calibración) y T3.8 (supervivencia) sobre más tuning. Dado que el modelo binario ya está diagnosticado como óptimo y sin margen por ajuste de hiperparámetros, el próximo salto de desempeño más probable vendrá de calibrar probabilidades (para decidir umbrales de contacto de marketing) y de aprovechar los clientes censurados (14.7%-15.8%) con un modelo de supervivencia, no de seguir tuneando LightGBM.

5. Extender el mismo diagnóstico a la regresión logística (Pipeline B). T3.2 mostró coeficientes con signo contra-intuitivo en n_visitas_12m y ticket_promedio (colinealidad con umbral_propio_dias). Una validation_curve sobre el parámetro C ayudaría a confirmar si ese problema es de regularización insuficiente o puramente de colinealidad estructural.

6.3 Limitaciones de este diagnóstico

• El régimen A (overfitting) y las 3 estrategias se evaluaron sobre un subconjunto de 3,000 filas de train, no sobre las 796,044 completas — necesario para inducir overfitting de forma didáctica; sus valores absolutos de PR-AUC no son directamente comparables con los de producción, aunque sí lo es la comparación relativa entre estrategias.

• El régimen de underfitting (num_leaves=2, learning_rate=0.001, 5 rondas) es un caso extremo construido a propósito; no representa ningún punto real ya explorado en _grid_lgbm_A2.json (todas las configuraciones probadas allí eran razonables).

• Se usaron log loss y PR-AUC (nunca accuracy, por la regla #4 del plan de acción del proyecto) para mantener consistencia con las métricas ya adoptadas en el resto del trabajo.

7. Referencias técnicas

• Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T. Y. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. Advances in Neural Information Processing Systems (NeurIPS) 30.

• Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830. (learning_curve, validation_curve).

• Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning* (2ª ed.), capítulo 7: Model Assessment and Selection. Springer.

• Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*, capítulo 7: Regularization for Deep Learning. MIT Press.

• Saito, T., Rehmsmeier, M. (2015). *The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets*. PLOS ONE 10(3). (justifica el uso de PR-AUC sobre accuracy/ROC-AUC en este problema desbalanceado).

• Documentación interna del proyecto: CONTEXTO_PROYECTO.md, PLAN_DE_ACCION.md, contexto_esquema_tablas.md, contexto_indices_rendimiento.md, 03_desercion_modelado.ipynb, artefactos/_grid_lgbm_A2.json, artefactos/modelo_lgbm_A2.joblib, artefactos/_tabla_final_test.csv.

Reporte generado automáticamente a partir de la ejecución completa y validada de `overfitting_analysis.ipynb`. Todas las cifras citadas provienen directamente de esa ejecución; ninguna fue estimada o inventada.

Validación Final

¿El reporte cumple los 4 componentes obligatorios de la rúbrica?

[X] Tracking de Métricas (25%): MetricsTracker + lgb.record_evaluation, logloss y PR-AUC por ronda, train y validación — Sección 2.2 / 3.

[X] Curvas de Aprendizaje (30%): curvas básicas de los 3 regímenes + learning_curve y validation_curve (avanzadas, opcionales) — Sección 4.

[X] Diagnóstico Sistemático (25%): regla cuantitativa por gap final y pendiente de regresión lineal, aplicada a los 3 regímenes y al modelo de producción — Sección 2.4 / 3.

[X] Estrategias de Mejora (20%): 3 estrategias implementadas y comparadas (se exigían ≥2), con tabla antes/después y figura comparativa — Sección 5.